🧪 Molecules as Bitstrings

Lab 2 of 3 · pointing the same method at real chemistry

James Weaver · Qiskit Fall Fest 2026

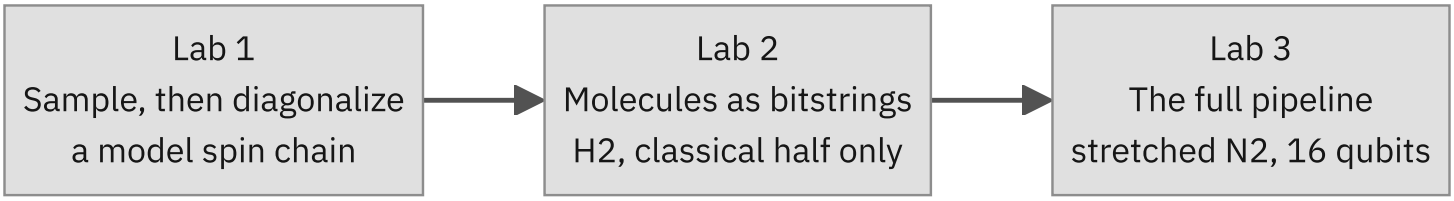
Background

In **May 2016**, IBM put a 5-qubit quantum processor on the cloud, the first one anybody could use from a browser.

A decade later, this series covers **sample-based quantum diagonalization (SQD)**, the method behind several of the largest quantum chemistry calculations run on quantum hardware so far.

> This session points that method at what it was built for: molecules.

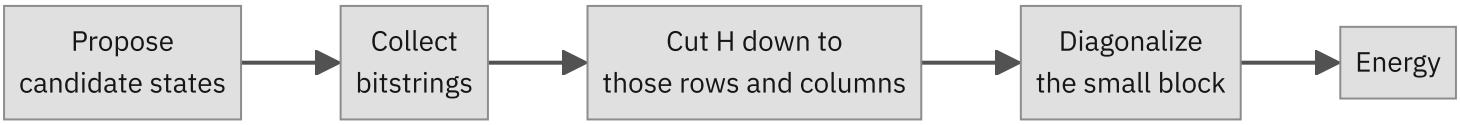
Where this lab fits



What you need from Lab 1: ground states of many systems concentrate on a small number of bitstrings, so you can cut the Hamiltonian down to just those and diagonalize the small block that is left.

No chemistry background is assumed. Everything you need is built up in the notebook.

The method does not change



The procedure is exactly Lab 1's. What changes is what a bitstring *means*, and where the Hamiltonian comes from.

> One thing worth saying plainly: **nothing quantum happens in this lab.** Today is about the chemistry representation and the classical half of the method. The quantum sampler shows up in Lab 3.

How Lab 1's ideas map onto chemistry

LAB 1	LAB 2
A row of magnets	A molecule (H_2)
An arrangement of 8 magnets	A pattern of electrons in orbitals (a **configuration**)
Bitstring = what the qubits measured	Bitstring = a *label* for one electron pattern
Hamiltonian from two simple rules	Hamiltonian from the molecule's **integrals**
Crop plus `np.linalg.eigh`	One call: `solve_fermion`
Sample from a circuit	Comes back in Lab 3

> Several things change at once here, which is why the notebook puts this table up front.

Electrons in orbitals

Here is all the chemistry vocabulary the lab needs.

- A molecule has a ladder of **orbitals**: places an electron can sit, ordered by energy. An orbital is a cloud of probability around the nuclei, not a little planetary track.
- Each orbital holds exactly **two electrons**, because of the **Pauli exclusion principle**: no two electrons can be in exactly the same state. Spin is what lets a pair share an orbital, so each gets one spin-up electron (**alpha**) and one spin-down (**beta**).
- A **configuration** is one specific pattern of which orbitals are filled.

> And here is the link back to Lab 1: **a configuration is just a bitstring.** One bit per orbital per spin, with `1` meaning filled and `0` meaning empty.

Where the molecule's Hamiltonian comes from

Nobody writes out a molecule's full Hamiltonian. Instead, a chemistry package hands you a compact set of numbers you can build any entry from on demand. It computes them from just the positions of the atoms and the name of a basis set.

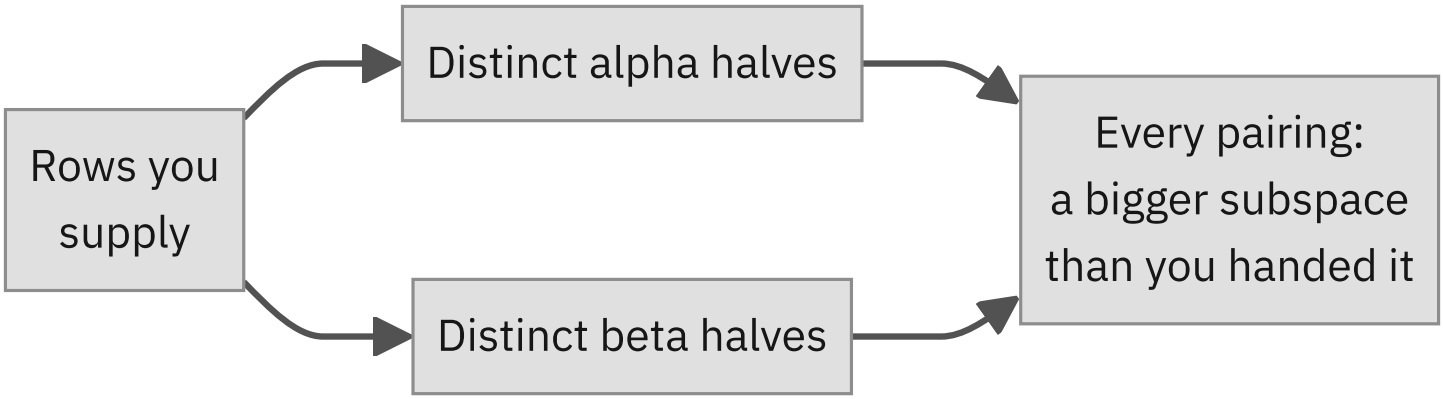
Chemists call those numbers the molecule's **integrals**, because each one is worked out by adding up contributions from every point in space where the electron clouds overlap. You will never compute one by hand, and the notebook treats that cell as boilerplate you can copy into any future project.

> They play the same role as Lab 1's two rules. One set says what it costs an electron to sit in an orbital or hop between orbitals; the other says how strongly two electrons push each other apart. Same structure, different rules.

The same crop, now for electrons

`solve_fermion` does what you wrote in Lab 1, but for electrons. Give it configuration rows plus the integrals and it returns the lowest energy you can reach inside that subspace.

One thing does differ from Lab 1, and the notebook spends real time on it. The solver splits each row into its alpha half and its beta half, and then uses **every alpha-beta pairing**.



> So the subspace ends up containing combinations you never supplied directly. The number of rows you provide and the size of the subspace you get are two different numbers, and the notebook keeps careful track of both.

Units, and how accurate we need to be

Energies come out in **Hartree**, the natural unit at atomic scale. One Hartree is roughly twice the energy it takes to pull hydrogen's electron off entirely, so it is a big unit for chemistry. The differences that matter are much smaller, so the working unit is the **milli-Hartree (mHa)**, one thousandth of a Hartree.

The target is **chemical accuracy**: get within **1.6 mHa** of the truth and predictions about reactions are generally trustworthy.

Hartree-Fock is chemistry's standard first approximation. It finds the single best pattern of filled orbitals and reports its energy. On H_2 it misses by about **25 mHa**, roughly 15 times too coarse.

> What it misses is called **correlation energy**: electrons actively getting out of each other's way. No single pattern of filled orbitals can capture that. Getting it requires a subspace of several configurations, which is exactly what we are building.

Pulling a bond apart

Hartree-Fock is not always 25 mHa off. Stretch a bond and it goes badly wrong.

Pull two hydrogen atoms apart and each one has to end up with its own electron. A single pattern of filled orbitals keeps the electrons paired up, so the energy it reports climbs away from the true curve.

> A **two-configuration** input, chosen well, tracks the true curve closely across the whole range of separations. Two configurations.

Bond stretching is the standard test where single-configuration methods break down and subspace methods hold up. It is the last thing you do in this lab.

What you will do

1. Write electron configurations as bitstrings and learn the packing convention.
2. Get a real molecule's integrals out of a classical chemistry package, in one cell.
3. Reproduce Hartree-Fock from a single configuration, then watch the energy drop as you add more.
4. See which configurations actually carry the weight: the chemistry version of Lab 1's concentration plot.
5. Stretch the bond and compare a small subspace against Hartree-Fock across the whole curve.

⏱ **About 60 minutes.** Almost none of that is waiting on cells; it is reading and thinking.

How the notebook works

Run the cells in order. You will see two markers throughout.

- 🧠 **Checkpoints** ask a question and hide the answer in a drop-down. Try to answer before you open it; that is where most of the learning happens.
- ✍️ **Your turn** cells ask you to write code between the `YOUR CODE HERE` markers. A worked solution is in a drop-down right underneath.

If you have not filled in an exercise yet, the notebook stops with a message containing a ✏️. That is the notebook reminding you, not something broken.

Three ways to run this lab

You get a single `.ipynb` file, and it is fully self-contained. There is no `requirements.txt` to fetch, no repository to clone, and no other files it depends on.

OPTION	WHERE IT RUNS	WHAT YOU INSTALL
Your own machine	Local Jupyter	Uncomment the `%pip install` line in the first cell
Google Colab	Browser, Google account	Uncomment the `%pip install` line in the first cell
qBraid Lab	Browser, free account	Build one environment, then leave that line commented

> You do not need an IBM Quantum account. Everything in this lab runs locally, so it uses no quantum processor time and no credits.

Options 1 and 2 · your machine, or Colab

On your own machine

```
python3 -m venv .venv
source .venv/bin/activate
pip install jupyterlab
jupyter lab
```

Then open the notebook and uncomment the `%pip install ...` line in its first cell, once.

- You need Python **3.11 to 3.14** on macOS or Linux.
- **On Windows, use WSL, Colab, or qBraid.** The chemistry package `pyscf` has no native Windows build.

On Google Colab

Upload the `.ipynb` file, then uncomment and run that same first-cell line. Nothing else changes.

Option 3 · qBraid Lab

A JupyterLab environment that runs in your browser, with a free tier. You build the environment once and reuse it.

1. Sign in at **lab.qbraid.com** (https://lab.qbraid.com).
2. Drag the `.ipynb` file into the file browser.
3. Open the **Environments** panel (upper right), then **Add** → **Create Environment**. Name it `fall-fest-sqd` and paste in this package list:

```
numpy>=2.3.2
qiskit==2.5.1
qiskit-aer==0.17.2
qiskit-addon-sqd==0.12.1
ffsim>=0.0.81
pyscf==2.14.0
matplotlib==3.11.1
pylatexenc==2.10
```

4. When **Installing…** turns into **Add kernel**, click **Add kernel**.
5. Open the notebook, pick the `fall-fest-sqd` kernel in the upper right, and leave the `%pip install` line commented. The environment already has everything.

Getting started

Open `02_molecules_as_bitstrings.ipynb` and run the first cell.

- The one-line summary of today: a configuration is just a bitstring.
- Watch the counting. Rows you supply and subspace size are different numbers.
- Nothing quantum today. The sampler comes back in Lab 3.

> 🧭 **If this feels like a stretch rather than a challenge,** the free Use a quantum computer today (https://quantum.cloud.ibm.com/learning/en/courses/use-a-qc-today) course on IBM Quantum Learning assumes no background and is a good place to start. This lab will still be here afterward.